00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 12

Açgözlü Algoritmalar

Yerel En İyi Seçim, En Kısa Yol ve En Küçük Kapsayan Ağaç

MODÜL IV · GRAFLAR VE AÇGÖZLÜ STRATEJİ

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** **Açgözlü Algoritmalar**	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan dört soru (yanıtlar notlarda)

- **S1 (H11).** `graf_ornek.txt` grafında BFS(0) ile 7 numaralı köşenin uzaklığı (kenar sayısı) kaç, yol hangisi?
- **S2 (H11).** 10 kenarlı yönsüz grafın komşuluk listelerinde toplam kaç öğe var?
- **S3 (H7).** 10 elemanlı en büyük öbeğe 97 eklenir ve köke kadar çıkar: kaç takas?
- **S4 (AP-1 H4).** {1, 5, 10, 20, 50, 100, 200} sisteminde 387 birimlik para üstü açgözlü yöntemle kaç parça?

TAHMİN ET

{1, 3, 4} sisteminde 6 birim: “her adımda sığan en büyük parayı al” kaç parça verir?

A 2 parça: 3 + 3 (açgözlü en iyiyi bulur)

B 3 parça: 4 + 1 + 1; en iyi 2 (3 + 3)

C 4 parça: 3 + 1 + 1 + 1

D 6 parça: 1 × 6

Büyük Soru: en iyi görünen seçim hep en iyi sonuca götürür mü?

{1, 3, 4} sisteminde 6: açgözlü 3 parça, en iyi 2 parça

Açgözlü: her adımda sığan en büyük para



= 6 → 3 parça

kalan 6: 4 × 1 → kalan 2 kalan 2: 1 × 2 → kalan 0

En iyi (kaba kuvvet)



= 6 → 2 parça

Bütün kombinasyonlar (4; büyük paradan küçüğe):

parça	3 (açgözlü)	2 (en iyi)	4	6
4·3·1 adet	1 · 0 · 2	0 · 2 · 0	0 · 1 · 3	0 · 0 · 6

Kanonik AP-1 sisteminde açgözlü 1–100 tutarın 100/100'ünde en iyi; {1, 3, 4}'te yalnız 76/100'ünde (Örnek 1, kaba kuvvetle)

GERÇEK DÜNYADAN

1956 · 1959

Yirmi dakikalık bir buluş: kâğıt kalem olmadan

- Dijkstra (2001 söyleşisi): bir sabah Amsterdam'da genç nişanlısıyla alışverişteyken, yorulup bir **kafe terasında** oturduklarında en kısa yol algoritmasını **yaklaşık yirmi dakikada** tasarladı.
- Algoritmanın güzelliğini **kâğıt kalem kullanmadan** tasarlanmasına bağlar; gösteri problemi: Rotterdam'dan Groningen'e en kısa yol.
- “Üç yıl sonra, **1959**'da yayımlandı”: buluş 1956'da; aynı yıl diploması ve ARMAC bilgisayarının açılışı.
- Üç sayfalık makale iki problem çözer: **en küçük kapsayan ağaç** ve **en kısa yol** — bu haftanın iki büyük konusu.

Frana, P. L., & Misa, T. J. (2010). An interview with Edsger W. Dijkstra. CACM, 53(8), 41–47; Dijkstra, E. W. (1959). Numerische Mathematik, 1, 269–271.

GERÇEK DÜNYADAN

1951 · 1952

Bir dönem ödevi: en verimli ikili kod

- Stix'e (1991) göre David A. Huffman **1951**'de MIT'de Robert M. Fano'nun bilgi kuramı lisansüstü dersindeydi.
- Öğrencilere **dönem ödevi** ile **final sınavı** arasında seçim hakkı verildi; ödev en verimli ikili kodu bulmaktı.
- Huffman'ın yöntemi **1952**'de yayımlandı: en seyrek iki simgeyi birleştir, tekrarla.
- Bugün: Cormen vd. tablosu ve bir Ömer Seyfettin metni üzerinde Huffman ağacı, kodlama, çözme ve entropi.

Stix, G. (1991). Encoding the “neatness” of ones and zeroes. Scientific American, 265(3), 54–58; Huffman, D. A. (1952). Proc. IRE, 40(9), 1098–1101.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 12.1	Açgözlü stratejiyi tanımlar; açgözlü seçim özelliği ve en iyi alt yapı kavramlarını açıklar ve açgözlü yaklaşımın başarısız olduğu durumlar için karşı örnek kurar.
ÖÇ 12.2	Aralık çizelgeleme ve kesirli sırt çantası problemlerini açgözlü yöntemle çözer; doğruluğu değiş tokuş argümanıyla gerekçelendirir.
ÖÇ 12.3	Huffman kodlamasını öncelik kuyruğuyla gerçekler ve sıkıştırma oranını hesaplar.
ÖÇ 12.4	Dijkstra algoritmasını öncelik kuyruğuyla gerçekler; negatif ağırlıklı kenarlarda neden başarısız olduğunu açıklar.
ÖÇ 12.5	En küçük kapsayan ağacı birleşim–bul destekli Kruskal algoritmasıyla bulur ve Prim algoritmasıyla karşılaştırır.

Açgözlü paradigma: iki koşul, üç kanıt tekniği

Ne zaman en iyi?

- **Açgözlü seçim özelliği:** ilk seçimi içeren **bir** en iyi çözüm vardır.
- **En iyi alt yapı:** kalan alt problemin en iyi çözümü + seçim = bütünün en iyisi.
- En iyi alt yapı tek başına yetmez: para üstü ve 0/1 sırt çantasında da var → H13 (DP).
- Kanıt yoksa: küçük girdilerde **kâhinle** (kaba kuvvet) karşılaştır, **karşı örnek** ara.

Nasıl kanıtlanır?

- **Açgözlü önde kalır** (greedy stays ahead): r'inci seçim, en iyinin r'incisinden geride değil.
- **Değiş tokuş argümanı** (exchange argument): en iyi çözümün parçası açgözlünün seçimiyle değiştirilir, çözüm kötüleşmez.
- **Kesim özelliği** (cut property): kesimi geçen en hafif kenar bir en küçük kapsayan ağaçtadır (Kruskal, Prim).
- Kleinberg ve Tardos (2006); Cormen vd. (2022).

Açgözlü ne zaman doğru, ne zaman yanlış?

Problem	Açgözlü ölçüt	En iyi?	Kanıt / karşı örnek	Örnek
Para üstü (AP-1 H4 sistemi)	en büyük para	✔ evet	100/100 tutarda en iyi	1
Para üstü {1, 3, 4}	en büyük para	✘ hayır	yalnız 76/100 tutarda	1
Aralık çizelgeleme	en erken biten ilk	✔ evet	değiş tokuş; 1000/1000	2
Aralık çizelgeleme	başka üç ölçüt	✘ hayır	karşı örnek; en kısa 953/1000	2
Aralık bölümleme	erken boşalan salon	✔ evet	salon = derinlik 1000/1000	3
Kesirli sırt çantası	birim değer	✔ evet	720 sıralamanın en iyisi	4
0/1 sırt çantası	birim değer	✘ hayır	160 < 220 (→ H13)	4
Önek kodu	Huffman: en küçük iki	✔ evet	B(T) = 224 bit	5
En kısa yol, w ≥ 0	Dijkstra: en küçük d	✔ evet	kâhin 3/3 graf	7–8
En kısa yol, w < 0 var	Dijkstra	✘ hayır	2 köşe yanlış	9
En küçük kapsayan ağaç	Kruskal / Prim	✔ evet	kaba kuvvet 280; 20/20	11–12
Graf boyama	Welsh–Powell	✘ hayır	taç: 4 renk, χ = 2	13

Haftanın problemleri: ölçüt, en iyi mi, kanıt ya da karşı örnek (bütün sayılar örneklerin çıktılarından)

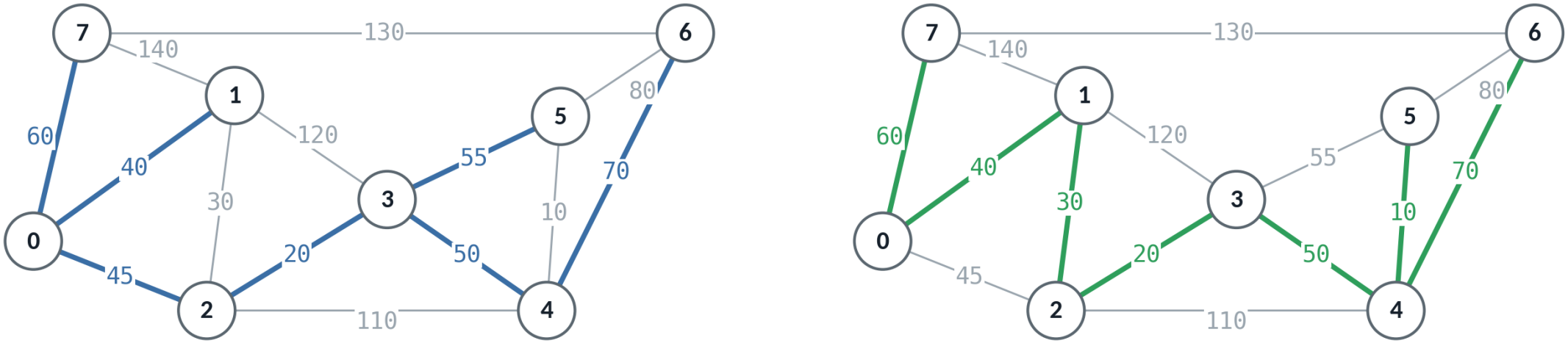
Haftanın grafı: aynı kampüs, iki farklı ağaç

En kısa yollar ağacı (Dijkstra, kaynak 0)

toplam ağırlık 340 m

En küçük kapsayan ağaç (Kruskal = Prim)

toplam ağırlık 280 m

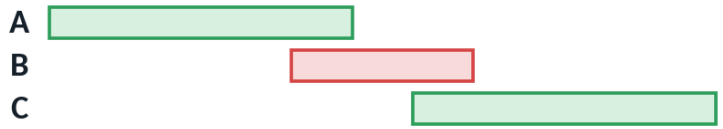


0 Ana Kapı · 1 Rektörlük · 2 Kütüphane · 3 Mühendislik · 4 Yemekhane · 5 Yurtlar · 6 Spor Salonu · 7 Otopark

Solda Ana Kapı'dan en kısa yollar ağacı (Dijkstra, toplam 340 m); sağda en küçük kapsayan ağaç (Kruskal = Prim, 280 m)

Aralık çizelgeleme: dört aday ölçüt, üçü yanlış

a) en kısa ilk



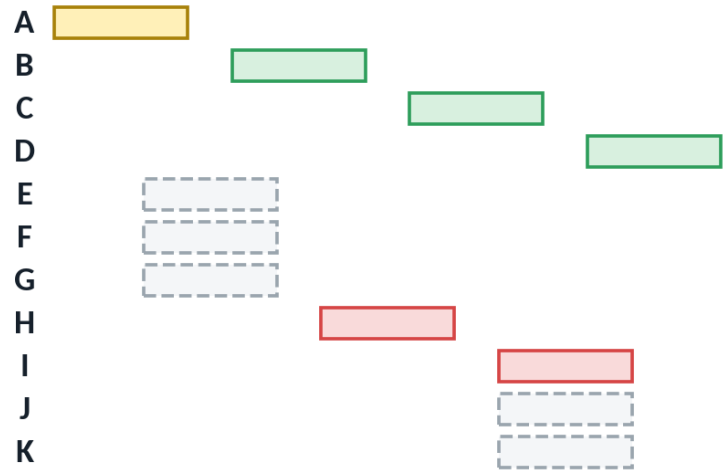
strateji: B → 1
en iyi: A C → 2

b) en erken başlayan



strateji: A → 1
en iyi: B C → 2

c) en az çakışan



strateji: H A I → 3
en iyi: A B C D → 4

kırmızı: yalnız stratejinin seçtiği · yeşil: yalnız en iyi çözümde · sarı: ikisinde de · kesikli gri: hiçbirinde

En kısa ilk, en erken başlayan ilk, en az çakışan ilk: kaba kuvvetle doğrulanmış karşı örnekler (kırmızı strateji, yeşil en iyi)

TAHMİN ET

B [0, 3) ve A [4, 7) seçildi; son bitiş 7. Sıradaki adaylar bitişe göre C [3, 8), G [6, 9), E [7, 10), F [9, 12). Hangisi seçilir?

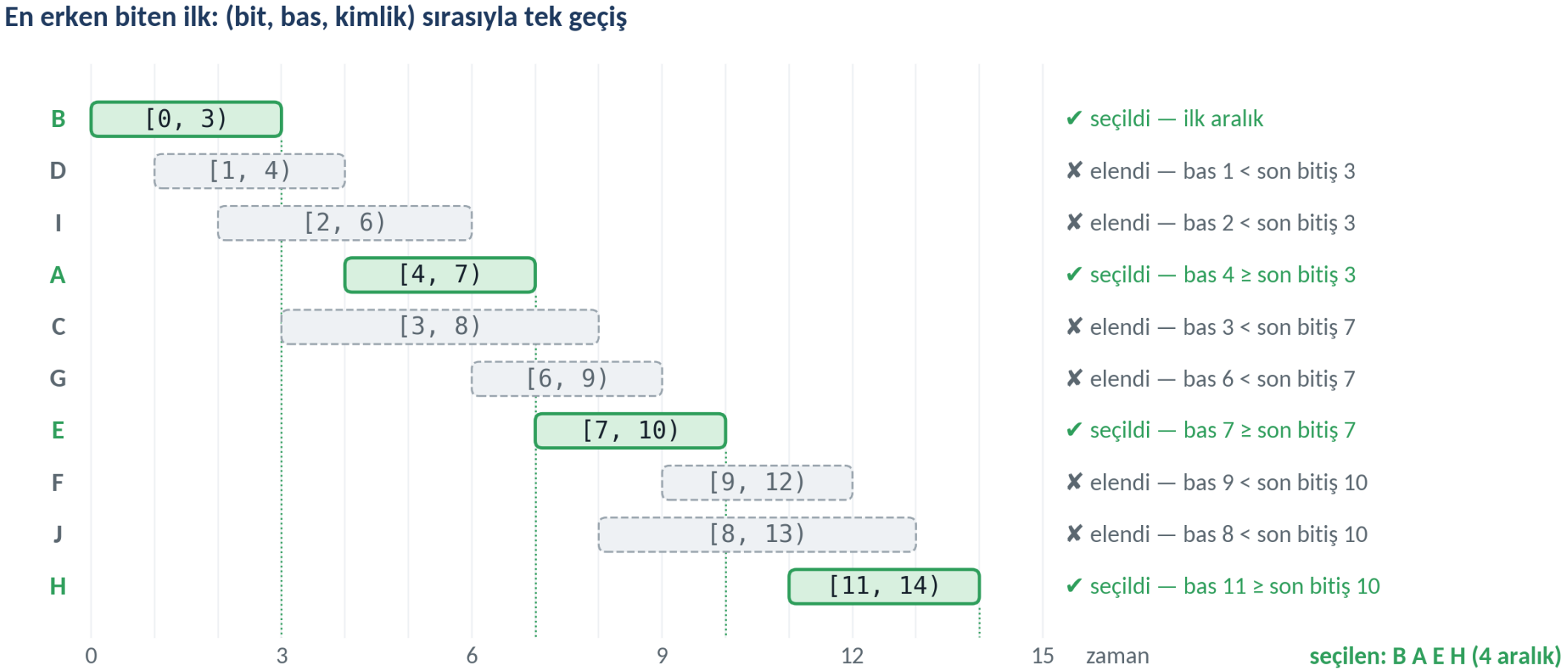
A C [3, 8)

B G [6, 9)

C E [7, 10)

D Hiçbiri: hepsi A ile çakışır

En erken biten ilk: tek geçiş, geri dönüş yok



Seçilen B, A, E, H (4 aralık; kaba kuvvet de 4); gri kesikli: elenen, noktalı: son bitiş

Sözde kod ve C: en erken biten ilk

```
algoritma EN-ERKEN-BİTEN(A)
  A'yı bitişe göre diz
  S ← boş;  son ← −∞
  her x için A'da (sırayla)
    eğer x.bas ≥ son ise
      S ← S ∪ {x};  son ← x.bit
  döndür S
```

kavram/H12_K05_cekirdek.c · satır 110–118

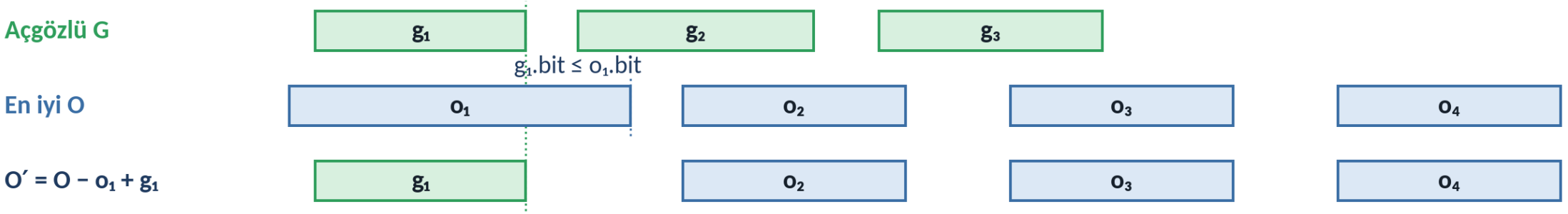
C

```c
qsort(a, (size_t)n, sizeof a[0], bitise_gore);
int k = 0, son_bitis = -1;
for (int i = 0; i < n; i++) {
    if (a[i].bas >= son_bitis) {     /* açgözlü seçim */
        secilen[k++] = a[i];
        son_bitis = a[i].bit;
    }
}
return k;
```

Θ(n log n) sıralama + Θ(n) geçiş · qsort kararsız: karşılaştırıcı tam sıralama (bit, bas, kimlik) tanımlar

Neden doğru? Değiş tokuş argümanı

Değiş tokuş argümanı: en iyi çözümün ilk aralığını açgözlününkiyle değiştir



1) g_1 bütün aralıklar içinde en erken bitendir: $g_1.bit \le o_1.bit$.
2) O′'de g_1, o_2'den önce biter ($o_2.bas \ge o_1.bit \ge g_1.bit$): O′ geçerli ve $|O'| = |O|$.
3) Kalan alt problem ($g_1.bit$'ten sonra başlayanlar) aynı yapıdadır: tümevarımla $|G| = |O|$.

Önde kalma: her r için $g_r.bit \le o_r.bit$ (Örnek 2: 1000/1000 rastgele örnekte denetlendi).

En iyi çözümün ilk aralığı g_1 ile değiştirilir: çakışma doğmaz, sayı değişmez (Örnek 2: 1000/1000 rastgele örnekte geçerli)

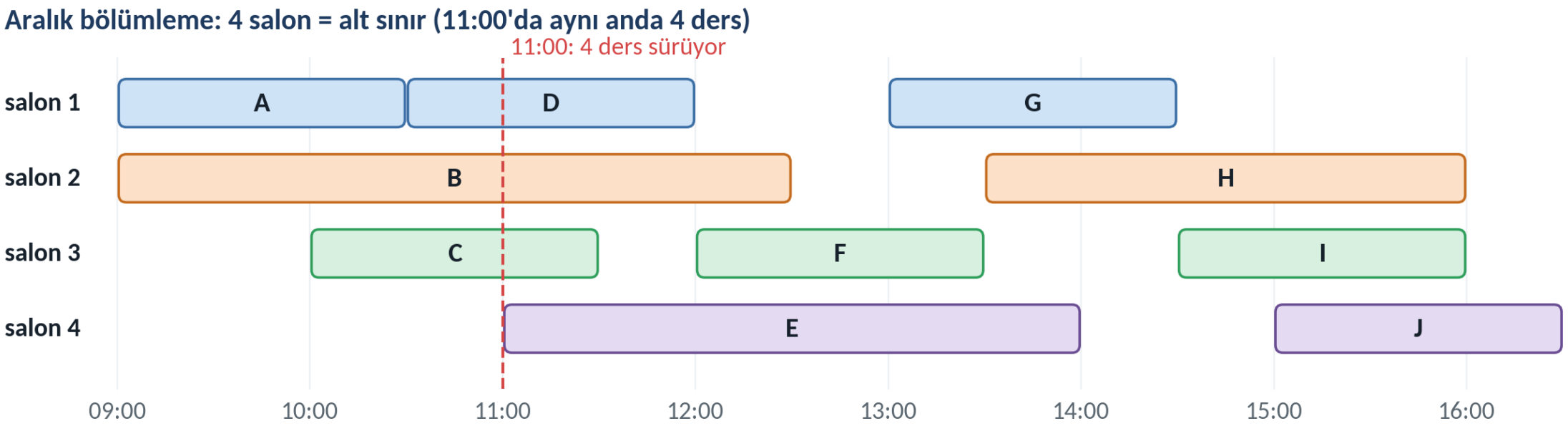
ALGORİTMA KİMLİK KARTI

Aralık çizelgeleme (en erken biten ilk) *interval scheduling, earliest finish time first*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Bitişe göre sırala; uyumluysa al	Yarı açık aralıklar [bas, bit)	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n)

Özellikler	Tek geçiş; seçim geri alınmaz; rastgele 1000 örnekte kâhinle aynı (Örnek 2)
Değişmez / doğruluk	Açgözlünün r'inci aralığı en iyinin r'incisinden geç bitmez (önde kalma)
Kütüphane karşılığı	C: qsort (seçim elle) · C++: std::sort + std::tie · Python: sorted(key=…)
Ne zaman kullanılır	Tek kaynak, en çok istek; ağırlıksız
Ne zaman kullanılmaz	Ağırlıklı aralıklar (H13: DP); bütün istekler (aralık bölümleme)

Aralık bölümleme: bütün dersler, en az salon



Başlangıca göre dolaş; öbekte salon bitişleri. 4 salon = derinlik (11:00'da 4 ders); rastgele örneklerde 1000/1000 eşit

Sözde kod: aralık bölümleme (öbekle)

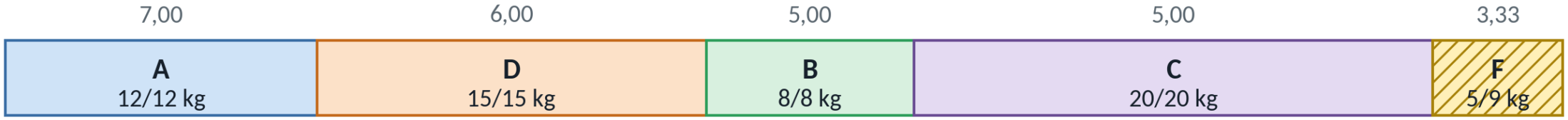
```
1  algoritma BÖLÜMLE(D)
2    D'yi başlangıca göre diz
3    Ö ← boş öbek;  k ← 0
4    her x için D'de (sırayla)
5      eğer Ö boş değil ve Ö.kök.bitiş ≤ x.bas ise
6        (_, s) ← ÇIKAR(Ö)          ▷ boşalan salon
7      değilse  s ← k;  k ← k + 1   ▷ yeni salon
8      salon[x] ← s;  EKLE(Ö, (x.bit, s))
9    döndür k
```

- **Analiz:** sıralama Θ(n log n) + her derste en çok bir çıkarma ve bir itme: O(n log n).
- **Kâhin:** derinlik, ayrı bir olay süpürmesiyle (aynı anda önce bitişler).
- **Yarı açık:** `<=` ile [0, 60) ve [60, 120) aynı salon; `<` yazmak hata (Mini-OBS 2 v2.11 test 7).
- **Varyant:** salon kapasitesi eklenince açgözlü artık en iyi değil.

Kesirli sırt çantası: birim değer; 0/1 sürümde yetmez

Kesirli: birim değere göre azalan sıra (A D B C F E), kapasite 60 kg

çubukların üstünde birim değer (değer/ağırlık); taralı yük kesirli alındı



toplam değer = 314 + 150/9 = 992/3 = 330,67

0/1 sürüm (kapasite 50): aynı ölçüt yetmez

A: 10 kg, 60	B: 20 kg, 100	C: 30 kg, 120

kesirli açgözlü: 240

0/1 açgözlü: AB → 160

0/1 en iyi: BC → 220

0/1 sırt çantası dinamik programlama ister (H13).

Kesirli: toplam 314 + 150/9 = 992/3 = 330,67 (değiş tokuşla en iyi) · 0/1: açgözlü AB → 160, kaba kuvvet BC → 220 (H13'e köprü)

TAHMİN ET

Huffman en seyrek iki ağacı birleştirir. Cormen vd. sıklıklarıyla ikinci birleştirmeden sonra öbek: 14:#6 16:d 25:#7 45:a. Üçüncü adımda hangi iki ağaç

A #6 + #7 → 39

B #7 + a → 70

C d + a → 61

D #6 + d → 30

Huffman: en seyrek iki ağacı birleştir

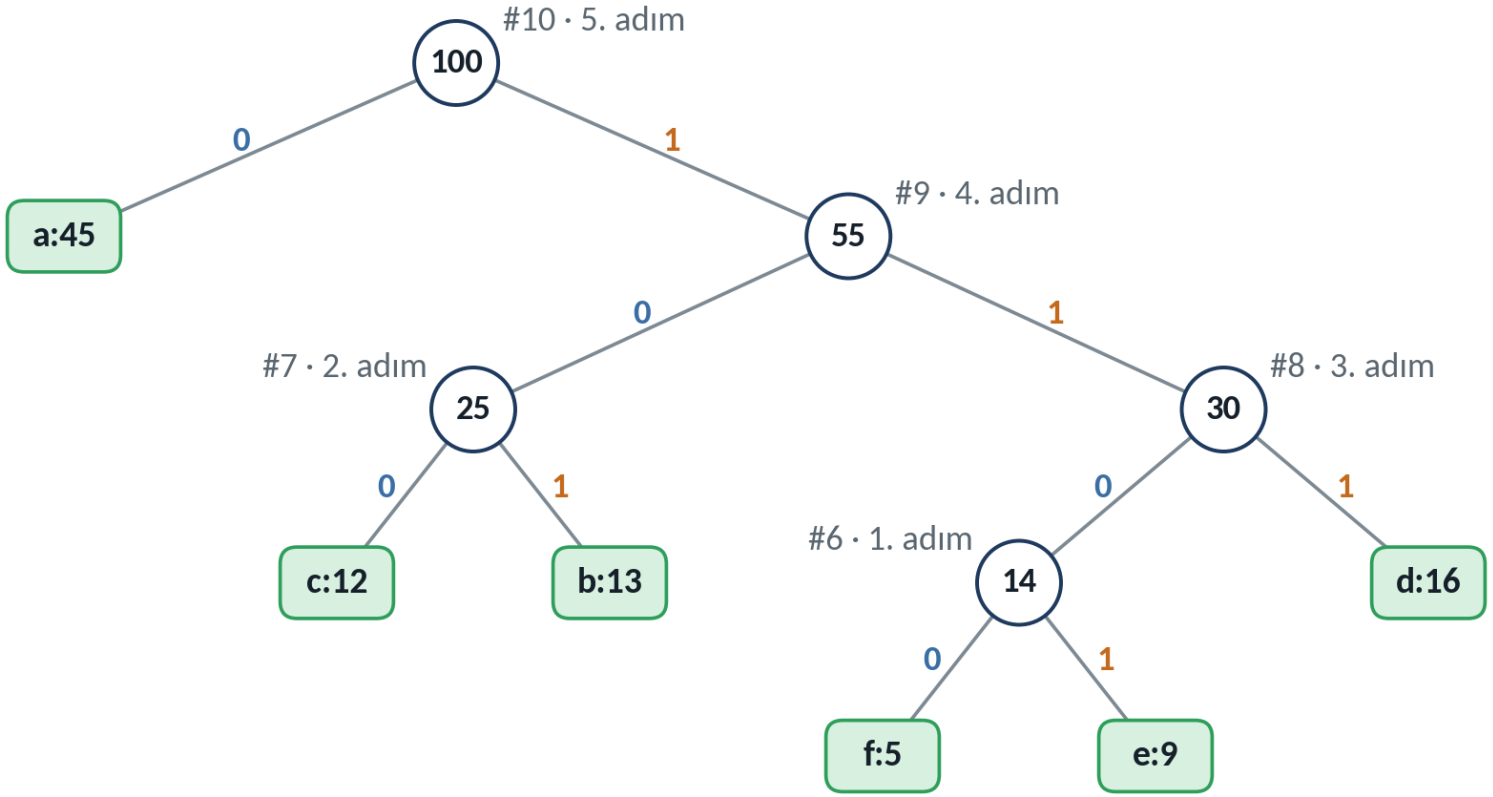
Huffman: öbekten en küçük iki ağacı çıkar (ilk → 0, ikinci → 1), toplamı geri it

başlangıç	`5:f` `9:e` `12:c` `13:b` `16:d` `45:a`	`5:f + 9:e → #6 (14)`
adım 1	`12:c` `13:b` `14:#6` `16:d` `45:a`	`12:c + 13:b → #7 (25)`
adım 2	`14:#6` `16:d` `25:#7` `45:a`	`14:#6 + 16:d → #8 (30)`
adım 3	`25:#7` `30:#8` `45:a`	`25:#7 + 30:#8 → #9 (55)`
adım 4	`45:a` `55:#9`	`45:a + 55:#9 → #10 (100)`
adım 5	`100:#10`	

sarı: sıradaki adımda çıkacak iki öğe · yeşil: az önce itilen iç düğüm · eşitlikte küçük düğüm no önce

Cormen vd. sıklıkları (simge sayısı olarak): öbekte (sıklık, düğüm no); sarı: sıradaki iki çıkarma, yeşil: yeni iç düğüm

Huffman ağacı ve kod tablosu



simge	sıklık	kod	bit
a	45	0	45
b	13	101	39
c	12	100	36
d	16	111	48
e	9	1101	36
f	5	1100	20

B(T) = Σ sıklık × uzunluk = 224 bit

sabit 3 bit/simge: 300 bit

B(T) = Σ sıklık × kod uzunluğu = 224 bit (sabit uzunlukta 300); B(T) iç düğüm sıklıklarının toplamına eşit

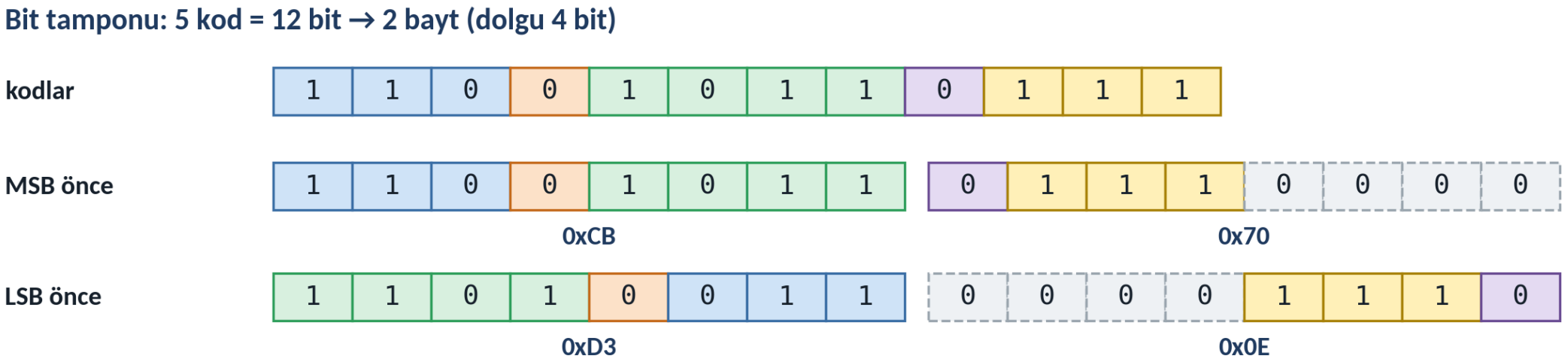
ALGORİTMA KİMLİK KARTI

Huffman kodlaması *Huffman coding*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
En seyrek iki ağacı birleştir (öbek)	Simge sıklıkları bilinir	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n)

Özellikler	n = farklı simge sayısı (metin uzunluğu değil); Cormen vd. tablosunda B(T) = 224 bit; tek simgede kod "0" (Örnek 5–6)
Değişmez / doğruluk	Öbekteki her düğüm bir ağacın kökü; kökün sıklığı yapraklarının toplamı
Kütüphane karşılığı	C: yok (elle öbek) · C++: std::priority_queue + std::greater · Python: heapq
Ne zaman kullanılır	Sıklıkları bilinen simgeler; önek kodu
Ne zaman kullanılmaz	Sıklıklar akışta değişiyorsa (uyarlamalı Huffman)

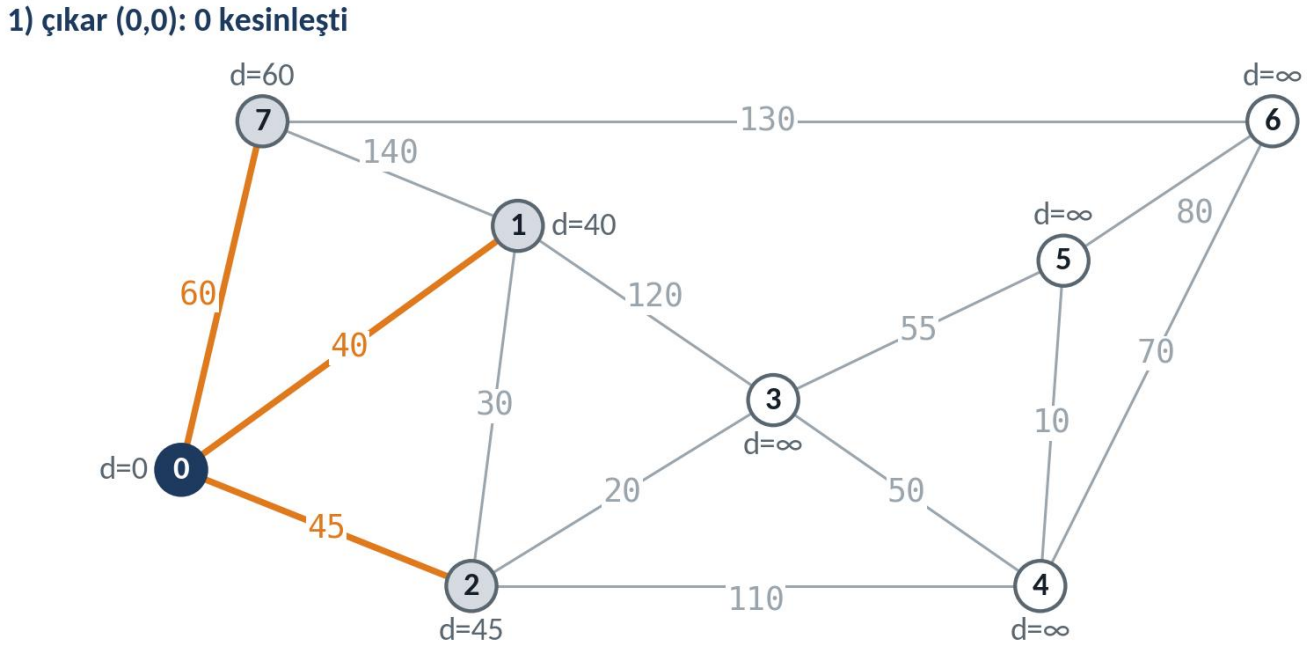
Kaputun Altında: kodlar bitlere nasıl yazılır?



Gri kesikli hücreler dolgu bitleridir (0). MSB önce: bit i, bayt i/8'in 7 − i mod 8 numaralı bitine (Örnek 6).
LSB önce: baytı en az anlamlı bitten doldurur (DEFLATE veri öğeleri böyle paketlenir; RFC 1951).

Bit i → bayt i >> 3; dolgu bitleri; çözme kökten yaprağa yürüyüş. Ömer Seyfettin metni (Örnek 6): %60,90, 4,8717 bit/simge (entropi 4,8462)

Dijkstra: kesinleşen küme dalga gibi büyür



öbek: (40,1) (45,2) (60,7)

1) çıkar (0,0): kaynak 0 kesinleşti, üç komşu gevşetildi

- Kesinleşmemişlerin **en küçük d'lisini** kesinleştir (açgözlü seçim).
- **Gevşetme:** d[u] + w < d[v] ise d[v] ← d[u] + w, ebeveyn[v] ← u.
- Öbekte (uzaklık, köşe) demetleri; azaltma yerine yeniden it (**tembel silme**).
- Kampüste 11 çıkarma, 11 itme, 3 eskimiş kayıt.

TAHMİN ET

4. çıkarmadan sonra öbek: (65,3) (155,4) (160,3) (190,6). Öbekten sıradaki köşe ve güncellenen uzaklıklar?

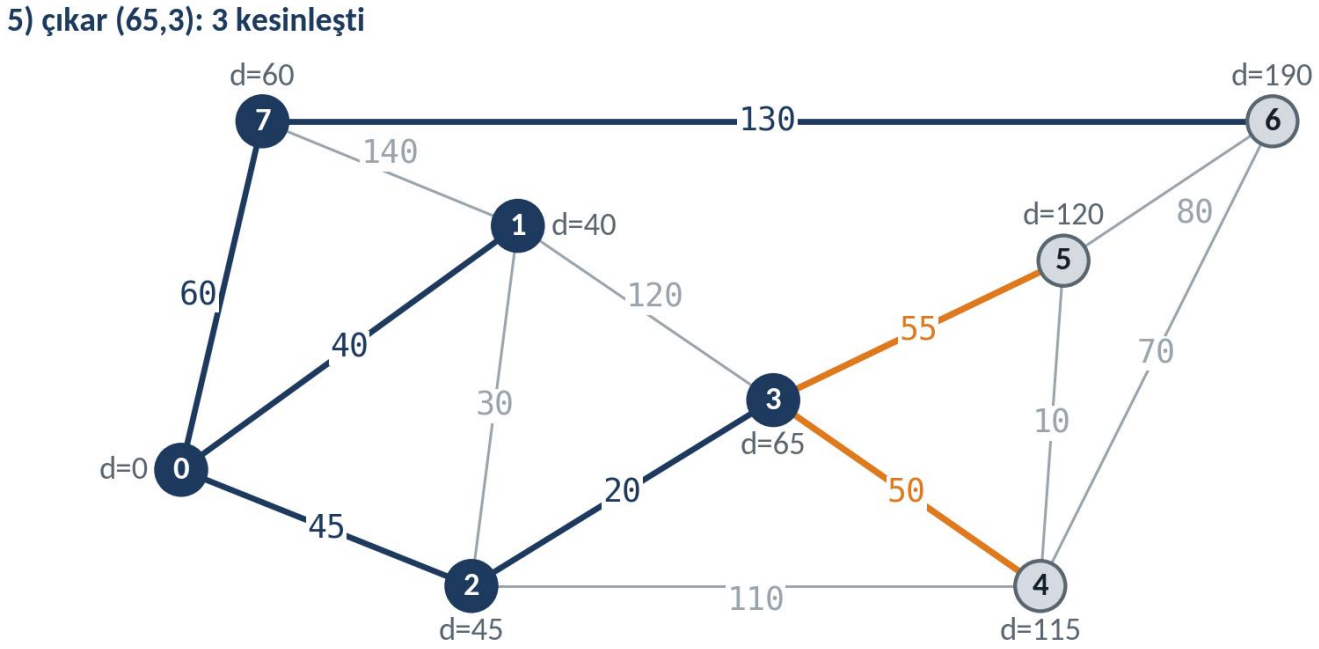
A (65,3): 3 kesinleşir; d[4] = 115, d[5] = 120

B (155,4): 4 kesinleşir; d[6] güncellenir

C (65,3): 3 kesinleşir; (160,3) öbekten hemen silinir

D (190,6): en uzak köşe önce kesinleşir

5. çıkarma: 3 kesinleşir, 4 ve 5 gevşer



öbek: (115,4) (120,5) (155,4) (160,3) (190,6)

öbek: (115,4) (120,5) (155,4) (160,3) (190,6) — 4 için iki kayıt

- d[4]: 155 (2 üzerinden) → 115 (3 üzerinden).
- Eski kayıt (155,4) öbekte kalır: **eskimiş**.
- Öbek en çok 1 + (başarılı gevşetme) ≤ E + 1 kayıt tutar.
- Köşe öbekten **çıkarken** kesinleşir, itilirken değil (Hata Avcısı 2).

TAHMİN ET

8. çıkarmada öbekten (155,4) çıkıyor. Ne olur?

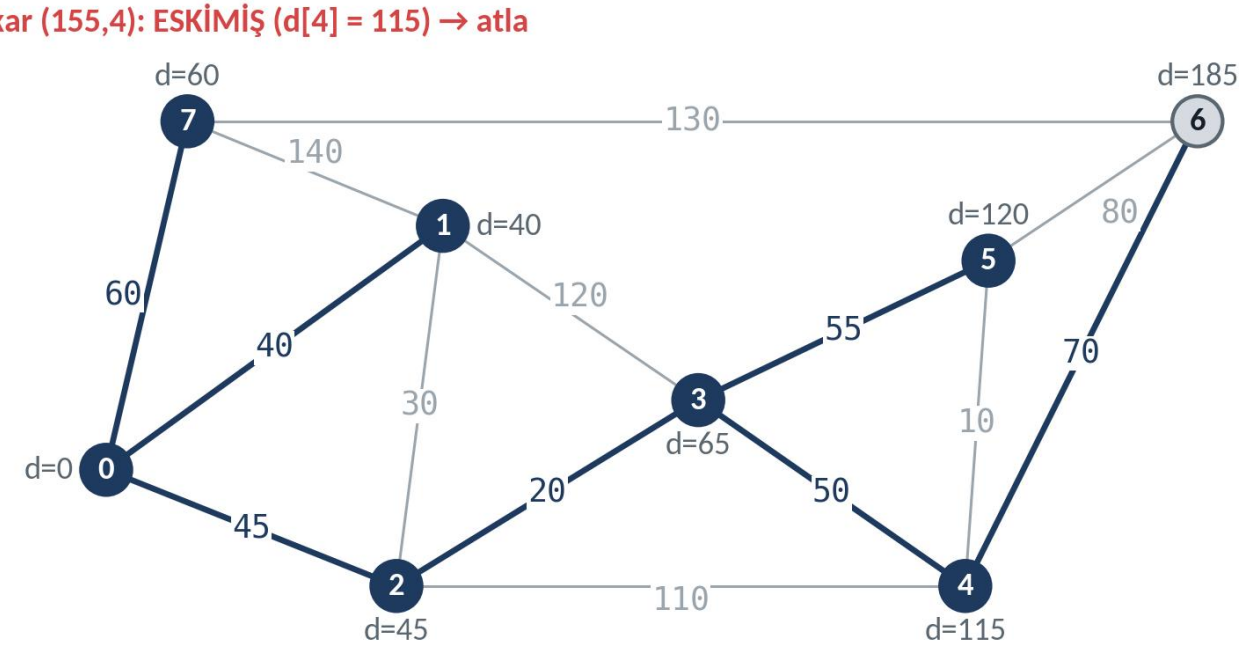
A 4 yeniden kesinleşir, d[4] = 155 olur

B Komşuları yeniden gevşetilir, d[6] düşer

C Kayıt atlanır: 4 zaten kesin, d[4] = 115

D Program hata verir: aynı köşe öbekte iki kez olamaz

8. çıkarma: eskimiş kayıt atlanır



öbek: (160,3) (185,6) (190,6)

Sonuç d = 0 40 45 65 115 120 185 60; 3 eskimiş kayıt atlandı

- Kampüste 11 itme = 1 + 10 başarılı gevşetme.
- Kenar inceleme 28 = 2E (yönsüz).
- En büyük öbek 5 kayıt.
- Ebeveyn dizisi en kısa yollar ağacını verir.

Öbekli Dijkstra: sözde kod ve C

```
algoritma DİJKSTRA(G, s)
  d[·] ← ∞;  d[s] ← 0
  Ö ← {(0, s)}
  iken Ö boş değil
    (du, u) ← ÇIKAR(Ö)
    eğer kesin[u] ise devam
    kesin[u] ← doğru
    her (u, v, w) için
      eğer d[u] + w < d[v] ise
        d[v] ← d[u] + w
        EKLE(Ö, (d[v], v))
```

kavram/H12_K05_cekirdek.c · satır 320–332 — C

```c
while (obek.n > 0) {
    int u = en_kucugu_cikar(&obek).no;
    if (kesin[u]) {
        continue;                    /* eskimiş kayıt */
    }
    kesin[u] = true;
    for (int i = 0; i < g->derece[u]; i++) {
        int v = g->komsu[u][i].hedef;
        int w = g->komsu[u][i].agirlik;
        if (!kesin[v] && d[u] + w < d[v]) {
            d[v] = d[u] + w;
            ebeveyn[v] = u;
            obege_ekle(&obek, (Oge){d[v], v});
```

Öbek O((V + E) log V) · dizi sürümü (Örnek 7) en küçük d'yi tarar: Θ(V²), kampüste 64 hücre · değişmez: kesinleşenlerde d gerçek uzaklık

Üç dilde: öbekten çıkar, eskimişi atla

H12_K05_cekirdek.c — C

```c
int u = en_kucugu_cikar(&obek).no;
if (kesin[u]) {
    continue;                    /* eskimiş kayıt */
}
kesin[u] = true;
```

H12_K05_cekirdek.cpp — C++

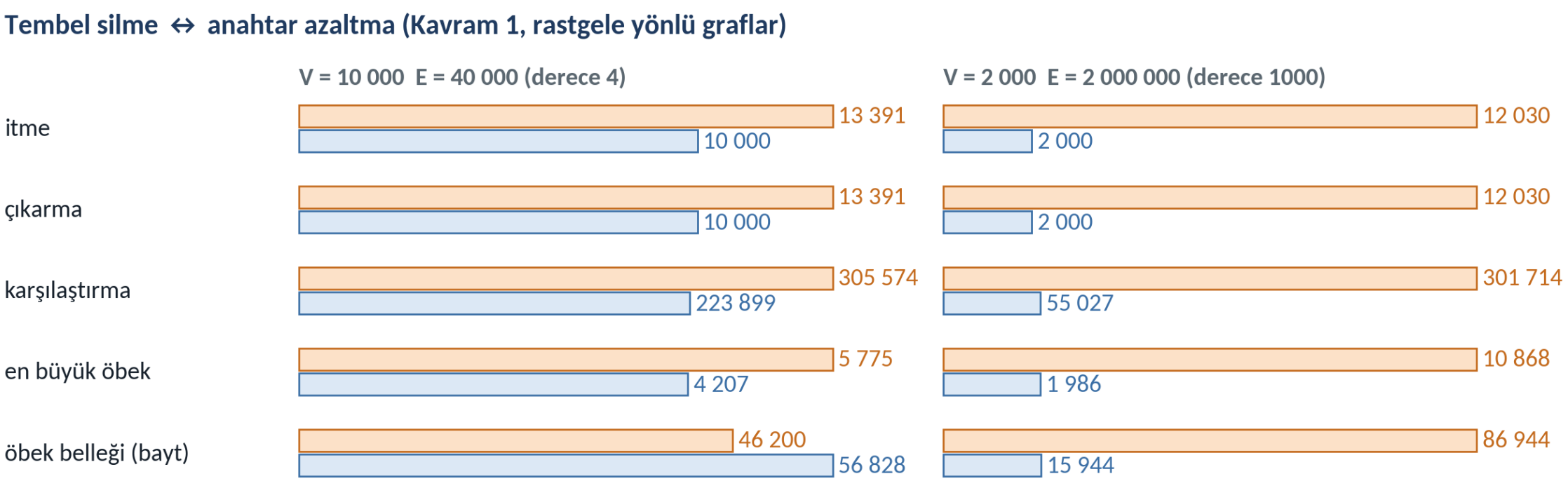
```cpp
auto [du, u] = obek.top();
obek.pop();
if (kesin[u]) {
    continue;                    // eskimiş kayıt
}
kesin[u] = true;
```

H12_K05_cekirdek.py — Python

```python
_, u = heapq.heappop(obek)
if kesin[u]:
    continue                     # eskimiş kayıt
kesin[u] = True
```

C: (anahtar, no) struct, en_kucugu_cikar elle · C++: top() + pop() iki işlem, yapılandırılmış bağlama · Python: heappop demeti döndürür (satırlar: C 321–325 · C++ 242–247 · Python 163–166)

Kaputun Altında: tembel silme ↔ anahtar azaltma



Kavram 1: yoğun grafta en büyük öbek 10 868 ↔ 1 986 kayıt; öbek belleği 86 944 ↔ 15 944 bayt

ALGORİTMA KİMLİK KARTI

Dijkstra (ikili öbek) *Dijkstra's algorithm, binary heap with lazy deletion*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
En küçük d'liyi öbekten çıkar, gevşet	Bütün ağırlıklar w ≥ 0	O((V + E) log V)	O((V + E) log V)	O((V + E) log V)	O(V + E)

Özellikler	Kampüste itme 11, atlanan 3, en büyük öbek 5 (Örnek 8)
Değişmez / doğruluk	Kesinleşen her u için d[u] gerçek en kısa uzaklık; öbekte d = d[v] olan kayıt güncel
Kütüphane karşılığı	C: yok (elle öbek) · C++: std::priority_queue (algoritma elle) · Python: heapq (algoritma elle)
Ne zaman kullanılır	Seyrek graflar, yol ağları
Ne zaman kullanılmaz	Negatif kenar (Bellman–Ford); yoğun ve küçük V'de dizi sürümü

TAHMİN ET

Yönlü grafta 0→1 (2), 0→2 (3), 1→3 (1), 2→1 (−2). Dijkstra 0'dan d[1] için ne bulur?

A 2: 1, 2'den önce kesinleşti; 2 → 1 (−2) kenarı geç kaldı

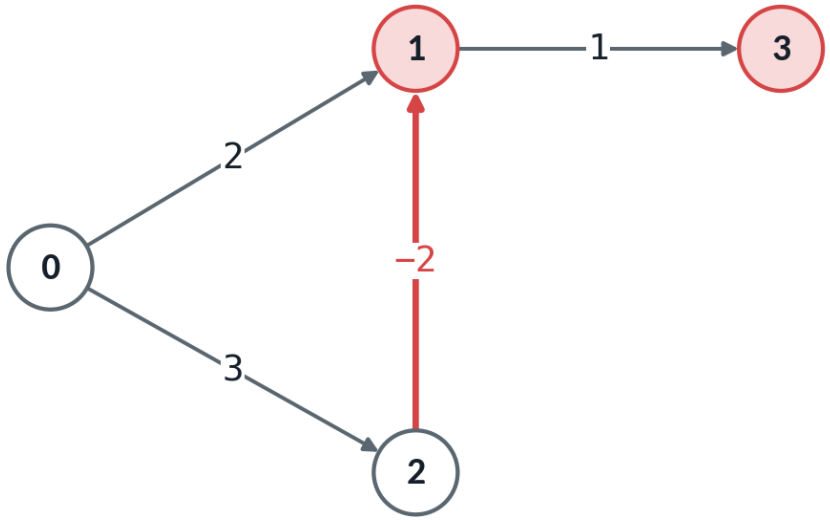
B 1: Dijkstra her zaman doğrudur

C −2: negatif kenar doğrudan d'ye yazılır

D ∞: negatif kenar 1'i ulaşılamaz yapar

Negatif kenar: Dijkstra'nın önkoşulu kanıtın parçasıdır

Negatif kenar: kesinleşen 1, sonradan bulunan kısa yolla güncellenemez



köşe	Dijkstra	Bellman–Ford	doğru?
0	0	0	evet
1	2	1	HAYIR
2	3	3	evet
3	3	2	HAYIR

ihlal: 2 → 1 (w = −2): d[2] + w = 1 < d[1] = 2

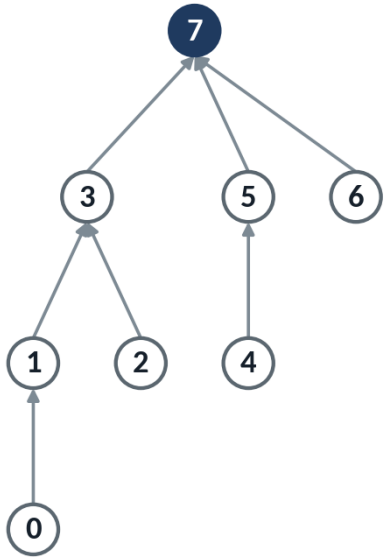
Bellman–Ford burada yalnız kâhindir (İleri Bakış).

2 köşe yanlış · "her ağırlığa 2 ekle" düzeltmesi de yanlış: 0 → 3 için 3 bulur, doğrusu 2

Birleşim–bul: rütbeyle birleştir, yolu sıkıştır

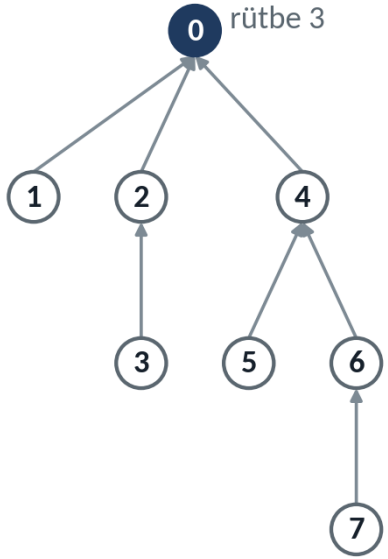
Birleşim–bul: n = 8; birleştir (0,1) (2,3) (0,2) (4,5) (6,7) (4,6) (1,5) (3,7); sonra bul(0)

saf



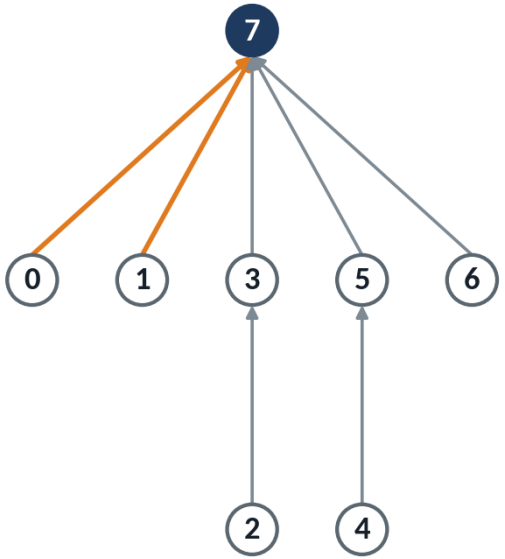
yükseklik 3 → 3
izleme 10 · bağlama 0

yalnız rütbe



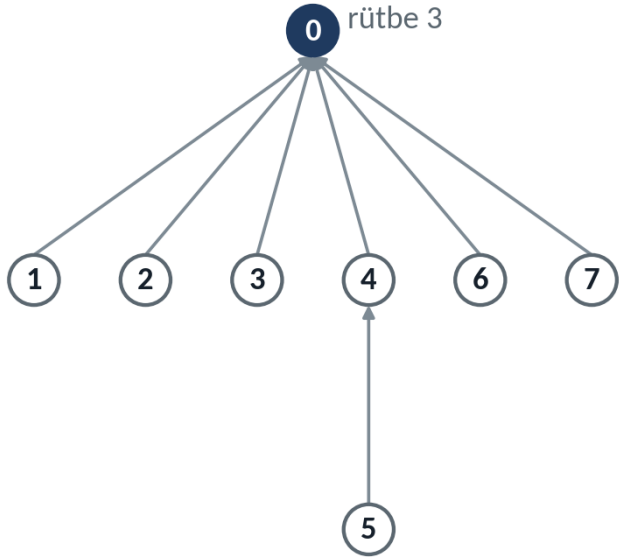
yükseklik 3 → 3
izleme 7 · bağlama 0

yalnız sıkıştırma



yükseklik 3 → 2
izleme 10 · bağlama 2

rütbe + sıkıştırma



yükseklik 2 → 2
izleme 7 · bağlama 3

siyah: kök · turuncu ok: bul(0) sırasında köke bağlanan (yol sıkıştırma) · ebeveyn dizileri bul(0)'dan sonraki hâldir

bul başına izleme ($n = 10^4 \rightarrow 10^6$): rütbe + sıkıştırma 0,7845 → 0,7905 → 0,7910; amortize $O(\alpha(n))$ (Tarjan, 1975), pratikte $\alpha \leq 4$ (Cormen vd., 2022)

Üç dilde: rütbeye göre birleştir

H12_K05_cekirdek.c — C

```c
if (b->rutbe[kx] < b->rutbe[ky]) {
    int g = kx;
    kx = ky;
    ky = g;
}
b->ebeveyn[ky] = kx;
if (b->rutbe[kx] == b->rutbe[ky]) {
    b->rutbe[kx]++;
```

H12_K05_cekirdek.cpp — C++

```cpp
if (b.rutbe[kx] < b.rutbe[ky]) {
    std::swap(kx, ky);
}
b.ebeveyn[ky] = kx;
if (b.rutbe[kx] == b.rutbe[ky]) {
    b.rutbe[kx]++;
```

H12_K05_cekirdek.py — Python

```python
if b.rutbe[kx] < b.rutbe[ky]:
    kx, ky = ky, kx
b.ebeveyn[ky] = kx
if b.rutbe[kx] == b.rutbe[ky]:
    b.rutbe[kx] += 1
```

Kök takası C'de geçici değişkenle, C++'ta std::swap, Python'da çoklu atamayla; false döndürmek = aynı küme (Kruskal'da döngü) (satırlar: C 373–380 · C++ 290–295 · Python 201–205)

TAHMİN ET

Kabul edilenler 4-5, 2-3, 1-2, 0-1, 3-4. Sıradaki (55, 3, 5): Kruskal bu kenarı kabul eder mi?

A Kabul: işlenmemiş en hafif kenar bu

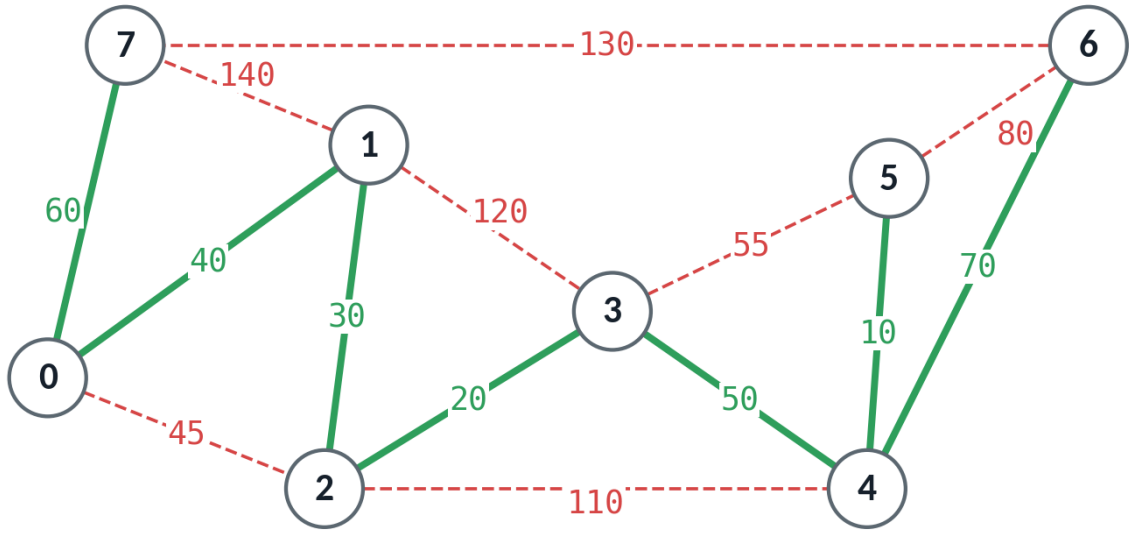
B Kabul: 5'in derecesi hâlâ 1

C Ret: 55 > 50, ağırlık artınca durulur

D Ret: 3 ile 5 aynı ağaçta (bul aynı kökü verir)

Kruskal: kenarları sırayla dene, döngü kuranı reddet

Kruskal (kampüs): kenarlar (w, u, v) sırasıyla; 7 kabul, toplam 280

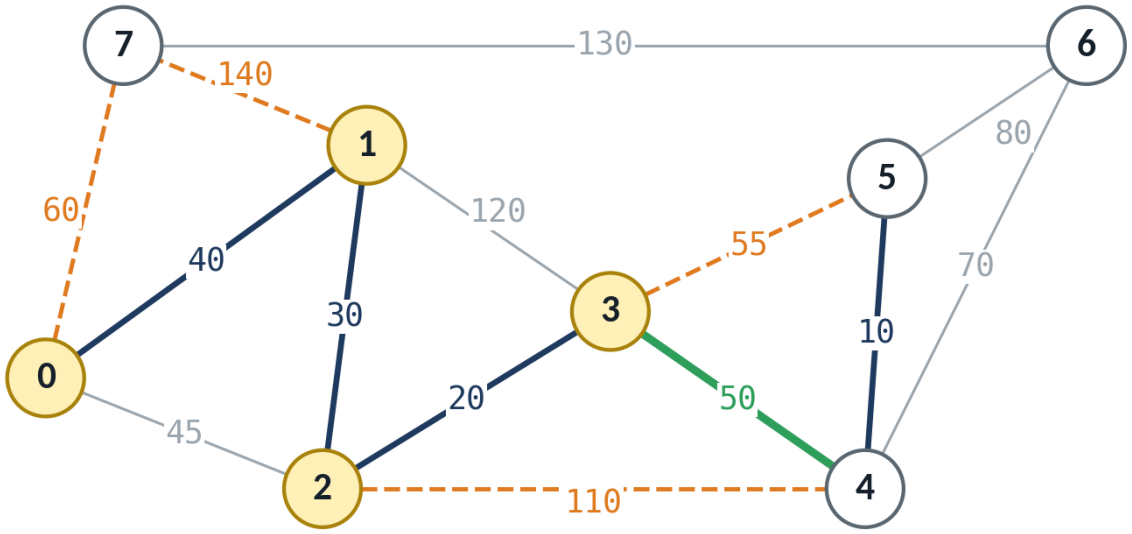


1.	(10, 4, 5)	**KABUL kökler 4, 5**
2.	(20, 2, 3)	**KABUL kökler 2, 3**
3.	(30, 1, 2)	**KABUL kökler 1, 2**
4.	(40, 0, 1)	**KABUL kökler 0, 2**
5.	(45, 0, 2)	RET: döngü kökler 2, 2
6.	(50, 3, 4)	**KABUL kökler 2, 4**
7.	(55, 3, 5)	RET: döngü kökler 2, 2
8.	(60, 0, 7)	**KABUL kökler 2, 7**
9.	(70, 4, 6)	**KABUL kökler 2, 6**
10.	(80, 5, 6)	RET: döngü kökler 2, 2
11.	(110, 2, 4)	RET: döngü kökler 2, 2
12.	(120, 1, 3)	RET: döngü kökler 2, 2
13.	(130, 6, 7)	RET: döngü kökler 2, 2
14.	(140, 1, 7)	RET: döngü kökler 2, 2

7 kabul, 7 ret, toplam 280; kâhin: 1 040 kapsayan ağacın en hafifi 280

Neden doğru? Kesim özelliği

Kesim özelliği (Kruskal 6. adım): S = {0, 1, 2, 3}



S (sarı)

kesimi geçen kenarlar: turuncu kesikli

en hafif geçen kenar: 3-4, w = 50

önceden kabul edilenler: lacivert

Kesimi geçen en hafif kenar

bir en küçük kapsayan ağaçtadır.

6. adım: S = {0, 1, 2, 3}; S'den çıkan en hafif kenar 3-4 (50) bir en küçük kapsayan ağaçtadır

TAHMİN ET

Prim 0'dan: ağaçta {0, 1, 2}; öbek (20,3) (45,2) (60,7) (110,4) (120,3). Sıradaki ağaç kenarı hangisi?

A (45,2): 0-2 kenarı (45)

B (20,3): 2-3 kenarı (20)

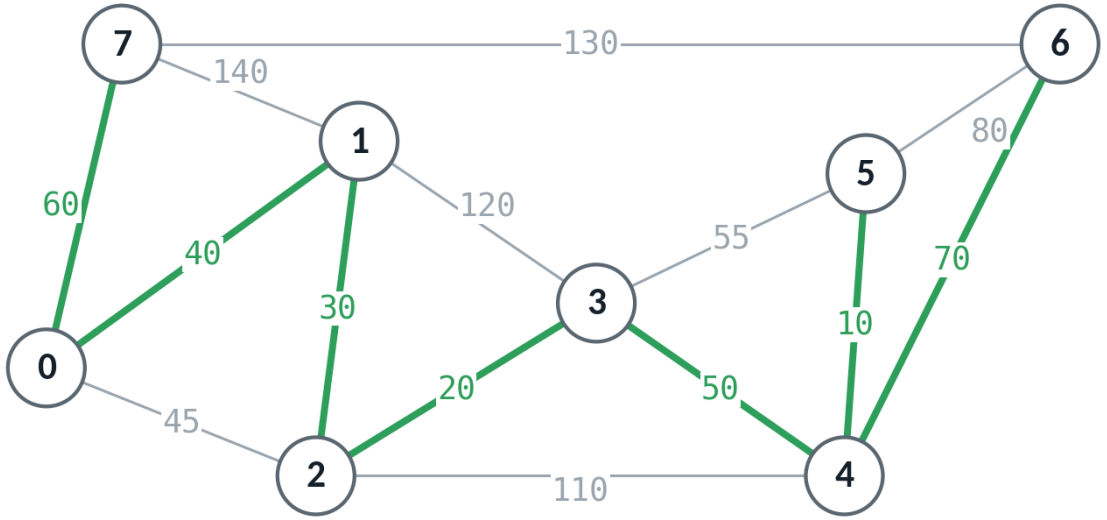
C (60,7): 0-7 kenarı (60)

D (110,4): 4'e giden kısa yol

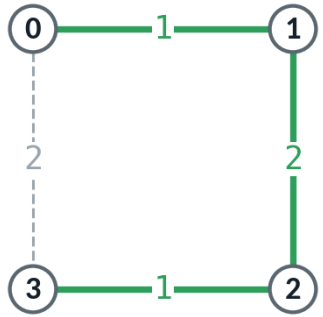
Prim: ağacı tek köşeden büyüt

Prim (0'dan): 7 kenar sırayla, toplam 280; eskimiş kayıt 4

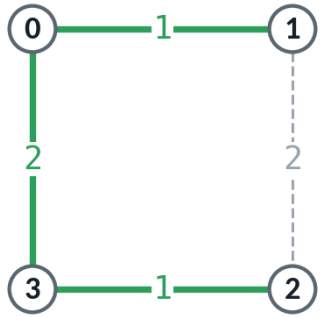
ağaca giriş sırası: 0-1 → 1-2 → 2-3 → 3-4 → 4-5 → 0-7 → 4-6



Eşit ağırlıklar (h12_esit_agirlik.txt):



Prim: toplam 4



Kruskal: toplam 4

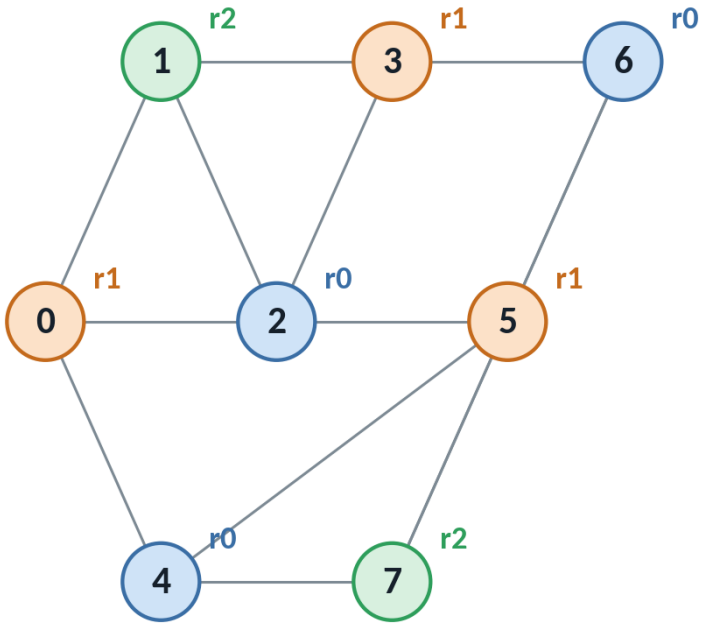
Kampüste toplam 280 = Kruskal; eşit ağırlıklı grafta iki farklı ağaç, aynı toplam (4)

Sekiz kimlik kartı bir bakışta (Kruskal ↔ Prim dahil)

Algoritma	Strateji	En kötü	Ek bellek	Ne zaman
Aralık çizelgeleme (en erken biten ilk)	Bitişe göre sırala; uyumluysa al	Θ(n log n)	O(n)	Tek kaynak, en çok istek
Kesirli sırt çantası	Birim değere (v/w) göre azalan sırala	Θ(n log n)	O(n)	Bölünebilir yükler
Huffman kodlaması	En seyrek iki ağacı birleştir (öbek)	Θ(n log n)	O(n)	Dengesiz simge sıklıkları
Dijkstra (dizi)	En küçük d'yi tara, kesinleştir, gevşet	$\Theta(V^2)$	O(V)	Kuramda yoğun graf, küçük V
Dijkstra (ikili öbek)	(uzaklık, köşe) öbeği; tembel silme	O((V + E) log V)	O(V + E)	Seyrek graflar, yol ağları
Birleşim–bul (rütbe + yol sıkıştırma)	Her küme bir ağaç; rütbe + yol sıkıştırma	O(α(n)) amortize	O(n)	Kruskal'da döngü denetimi
Kruskal	Kenarları (w, u, v) sırasıyla; döngüyü reddet	O(E log E)	O(V + E)	Kenar listesi hazır, seyrek graf
Prim (ikili öbek)	Ağaçtan çıkan en hafif kenar (öbek)	O(E log V)	O(V + E)	Komşuluk listesi; tek köşeden büyüt

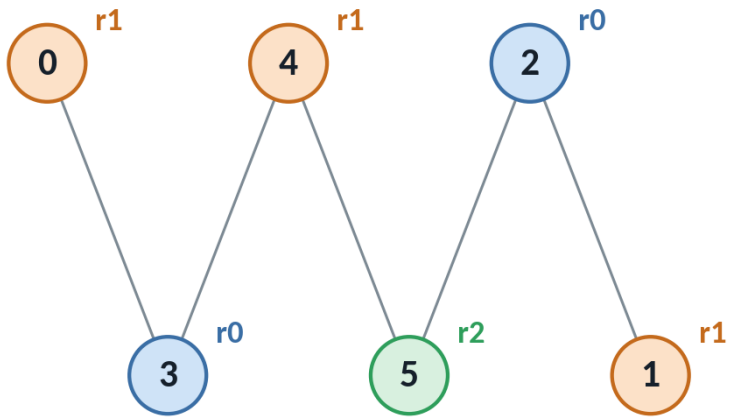
Açgözlü sezgisel: Welsh–Powell graf boyama

sınav çakışma grafı



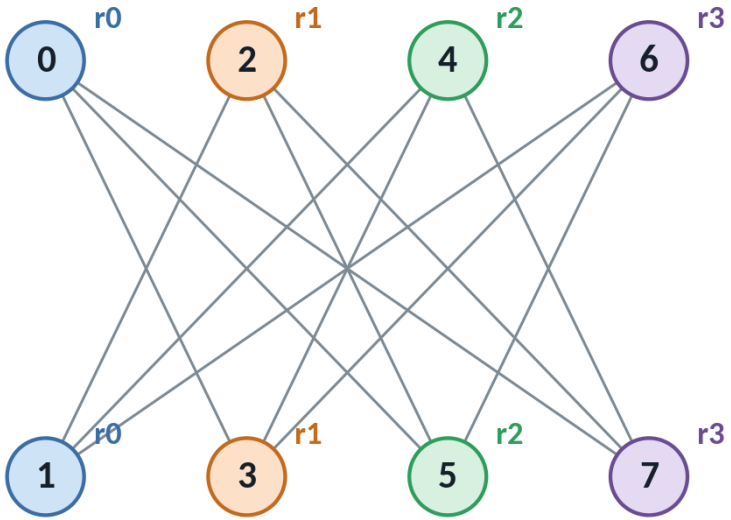
Welsh–Powell 3 renk · χ = 3

yol (6 köşe)



Welsh–Powell 3 renk · χ = 2

taç grafı (8 köşe)

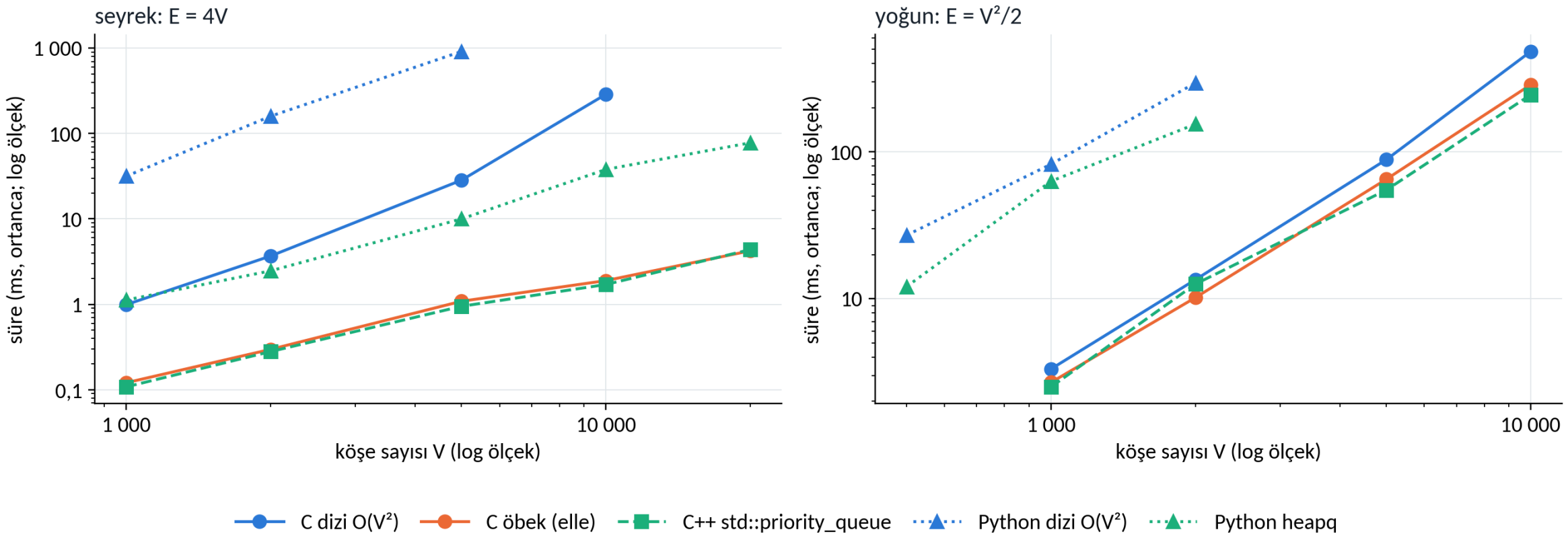


Welsh–Powell 4 renk · χ = 2

Sınav grafında 3 renk = χ; yol grafında 3 (χ = 2); taç grafında 4 (χ = 2): "en iyi" denmez

Haftanın Deneyi (a): dizi Θ(V²) ↔ öbek O((V + E) log V)

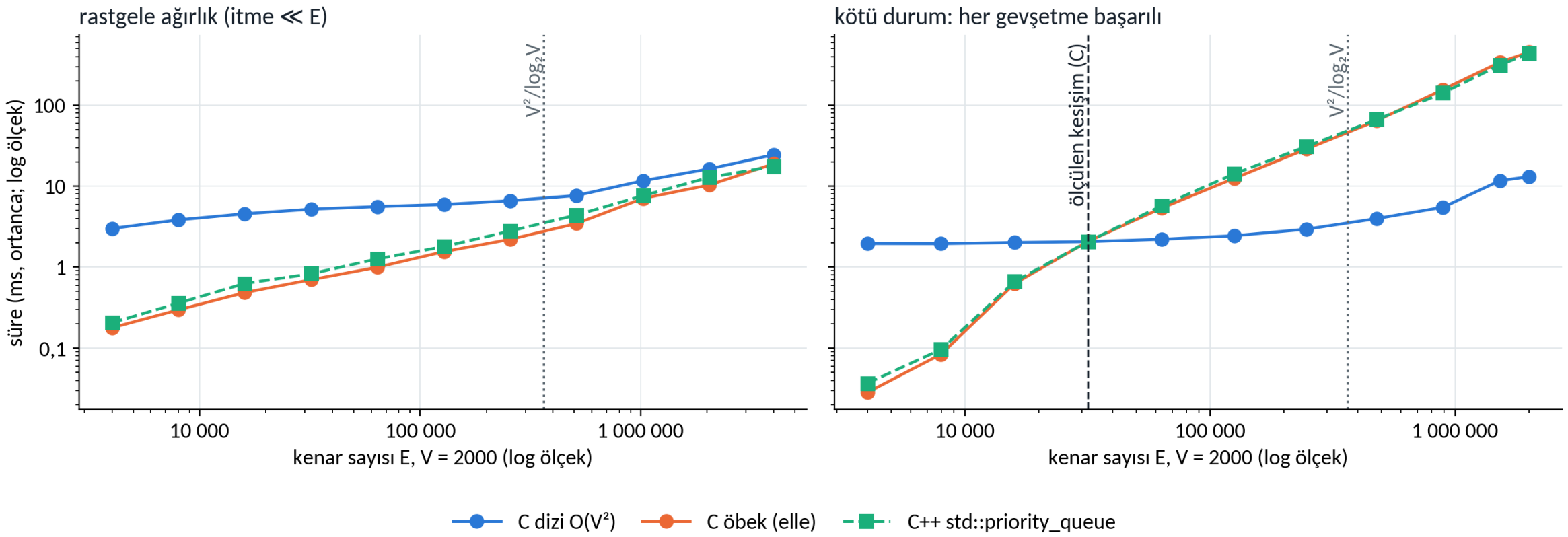
rastgele yönlü graflar, ağırlık 1..1000 (LCG, tohum 42); 1 ısınma + 5 tekrar, ortanca (bu makinede)



Seyrek (E = 4V): V = 10⁴'te dizi ÷ öbek 152 kat · yoğun (E ≈ V²/2): 1,23–1,68 kat (bu makinede)

Haftanın Deneyi (b): kesişim nerede? Kuram ne kadar yanılır?

aynı V'de yoğunluk taraması; kesikli dikey = ölçülen kesişim, noktalı = kuramsal kestirim (bu makinede)



Rastgele ağırlık: kesişim YOK (itme/E = 0,0032) · kötü durum: E ≈ 31 717, kuramsal 364 771 (11,5 kat)*

Hata Avcısı: altı hata, bu makinede gözlenen belirtiler

#	Hata	Belirti (gerçek çıktı)	Çözüm
1	Yanlış ölçüt: en kısa aralık ilk	[(0, 5), (4, 7), (6, 11)]: 1 aralık, kâhin 2	Bitişe göre; kâhinle rastgele sına
2	Köşeyi öbekten çıkarmadan kesin saymak	d[3] = 160 (65), d[4] = 155 (115), d[5] = 165 (120), d[6] = 190 (185)	Öbekten ÇIKINCA kesinleştir
3	Eskimiş kaydı atlamamak	sonuç doğru; yoğun grafta kenar inceleme 2 230 452 (doğrusu 499 951)	if (kesin[u]) continue;
4	SONSUZ + w taşması	UBSan: signed integer overflow; d[4] = -2147483647	Seçimde ve gevşetmede d ≠ SONSUZ
5	Kruskal'da döngü denetimi yok	7 kenar, toplam 250 < 280; iki köşe bağlanmadı	bul(u) ≠ bul(v) ise kabul
6	Huffman'da tek simgeli metin	C: 0 bit, çözülen boş; Python: IndexError: bytearray index out of range	Tek simgede kod "0"

Hata Avcısı 2: köşe öbekten çıkınca kesinleşir

hata_avcisi/H12_HA2_erken_kesinlestirme.c · satır 89–93 (hatalı) — C

```c
if (!kesin[v] && d[u] + w < d[v]) {
    d[v] = d[u] + w;
    ebeveyn[v] = u;
    kesin[v] = true;     /* HATA: çıkarılmadan kesin */
    obege_ekle((Oge){d[v], v});
```

hata_avcisi/H12_HA2_erken_kesinlestirme_duzeltilmis.c · satır 82–86 — C

```c
int u = en_kucugu_cikar().kose;
if (kesin[u]) {
    continue;                    /* eskimiş kayıt */
}
kesin[u] = true;                 /* DÜZELTME: çıkarılınca kesin */
```

HATALI: UZAKLIKLAR

köşe 3: 160 köşe 4: 155 köşe 5: 165 köşe 6: 190

DÜZELTİLMİŞ

köşe 3: 65 köşe 4: 115 köşe 5: 120 köşe 6: 185

Dil Penceresi: öncelik kuyruğu, sonsuz, demetler, bitler

	C	C++	Python
Öncelik kuyruğu	H7 öbeğinin genellemesi (struct + kucuk_mu)	priority_queue<pair<int,int>, vector<…>, greater<…>>	heapq: (uzaklık, köşe) demetleri
Varsayılan yön	(elle)	en BÜYÜK öbek (std::less)	en KÜÇÜK öbek
Demet karşılaştırma	struct + karşılaştırıcı	std::pair / std::tie sözlük sırası	demetler sözlük sırası; Dugum ise TypeError
Sonsuz	INT_MAX / LLONG_MAX; taşma koruması	numeric_limits<int>::max()	math.inf (float) ya da aynı sabit
Bit tamponu	uint8_t dizisi (calloc)	std::vector<std::uint8_t>	bytearray; int.to_bytes
Anahtar azaltma	konum dizisiyle elle	yok: tembel silme	yok: tembel silme

Mühendis Gözüyle · Atölyeler · İleri Bakış · Merak

Mühendis Gözüyle

- Açgözlü çözüm **ya kanıtlanır ya da karşı örnekle sınanır**; sezgisel sonuç “en iyi” diye raporlanmaz.
- “En kısa ilk” rastgele örneklerin 479/500'ünde doğruydu: rastgele test kanıt değildir.
- Önkoşulu program denetlesin (Dijkstra'da $w \geq 0$).

Atölyeler

- Açgözlü mü, değil mi? On problem kartı: strateji öner, karşı örnek ara.
- Huffman: kendi adın ve bir paragraf (elle ağaç + programla doğrulama).
- Kampüs haritası (eşli): Dijkstra ve en küçük kapsayan ağaç (fiber).

İleri Bakış

- **H13:** {1, 3, 4} para üstü ve 0/1 sırt çantası DP ile. Bellman–Ford, Floyd–Warshall.
- **Navigasyon:** A* (d[u] + h(u)) ve ön işlemeli yöntemler; DEFLATE (LZ77 + Huffman): ZIP, PNG; entropi.
- **Hazırlık (H13):** ders notu Bölüm 4.3 ve 13; Alıştırma 12.1–12.8; web oynatıcıları.

Merak: matroidler

- Bağımsız kümeler + değiş tokuş özelliği → ağırlığa göre açgözlü en iyiyi bulur.
- Ormanlar bir matroiddir: Kruskal. Çakışmasız aralık kümeleri bir matroid mi?

Mini-OBS 2 v2.11: sınav programı

164,9 ms

Welsh–Powell, 10^6 derslik çakışma grafı ($E \approx 4 \cdot 10^6$); okunan komşu = 2E

414,2 ms

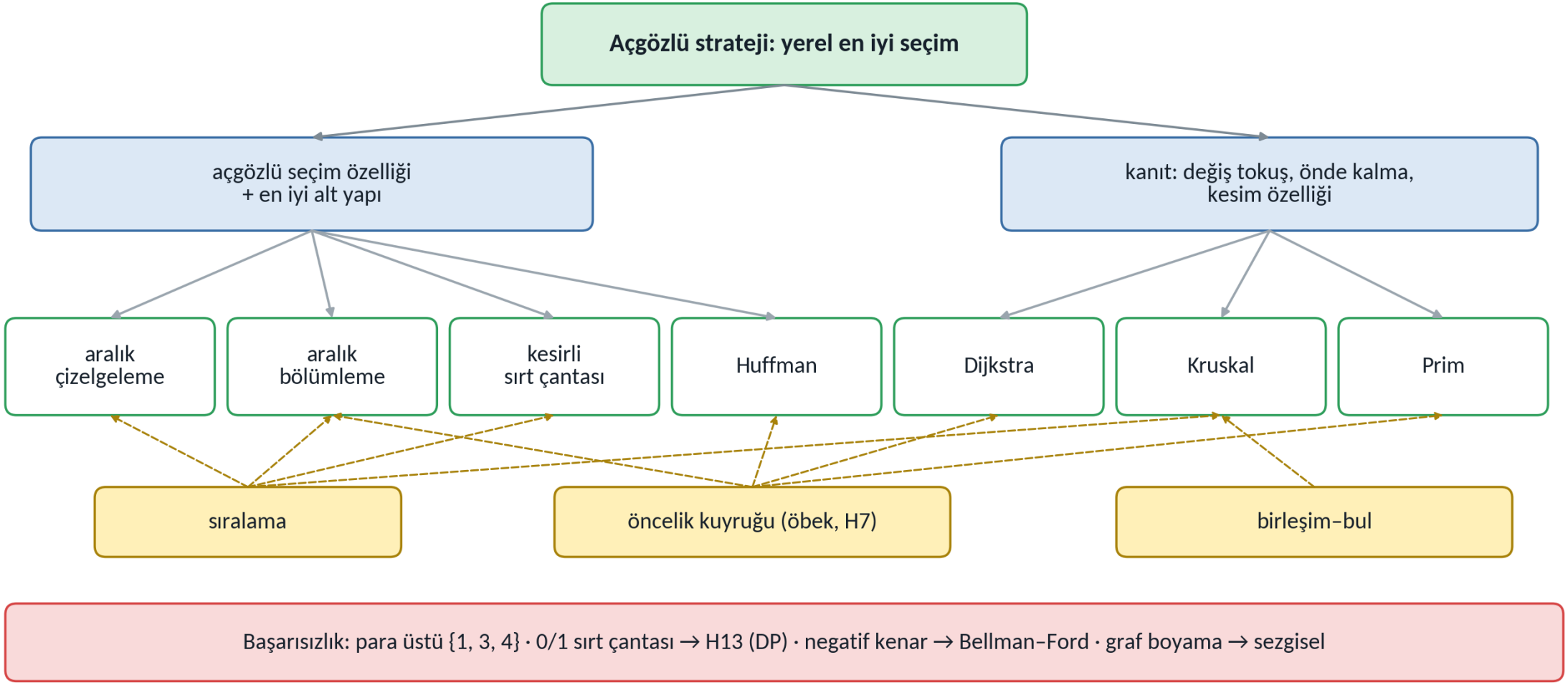
aralık bölümleme, 10^6 sınav; her n'de salon = derinlik

%88,0–%99,7

n = 10 derste Welsh–Powell = χ oranı (rastgele sıra: %60,5–%87,7)

- Çakışma grafı (karma tablosuyla) → **Welsh–Powell** oturumları + klik alt sınırı; ≤ 30 derste kaba kuvvetle χ. Her oturumda **aralık bölümleme** ile en az salon.
- 8 kabul ölçütü, 12 test senaryosu (v2.11 belgesi § 10); taç grafında açgözlü 4, χ = 2: çıktıda “en iyi” sözcüğü kullanılmaz.

Özet: açgözlü strateji, kanıtlar, araçlar, sınırlar



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. {1, 5, 12} sisteminde 15 birim: açgözlü kaç parça verir, en iyisi kaç? Hangi koşul bozuluyor?

2. Kampüs grafında Dijkstra kaynak 7'den (Otopark) başlarsa ilk kesinleşen üç köşe ve uzaklıkları?

3. Öbekli Dijkstra'da öbekten çıkan eskimiş bir kayıt neden atlanabilir? Atlanmazsa sonuç değişir mi?